CIFP CIUDAD JARDÍN LHII / DM 2 / ADAT

METODOLOGÍAS ÁGILES · GUÍA PARA PROYECTOS DE SOFTWARE

Planificación, Scrum, Kanban y colaboración · ADAT · Caso de aplicación: Reto 2

APLICACIÓN AL RETO 2 · DOS ENTREGAS OBLIGATORIAS

HITO 1 - Contrato técnico en PDF con el **ANEXO I** incorporado. Entrega en Moodle al finalizar Discovery.

HITO 2 - **ANEXO II** de seguimiento en PDF con la documentación final; software en su entrega de Moodle. Al finalizar Sprint 2.

Esta guía será una referencia de trabajo durante todo el curso. En el Reto 2 aprenderéis a planificar, desarrollar y revisar por iteraciones; en retos posteriores aplicaréis estas prácticas con más autonomía y, cuando proceda, con feedback del cliente y cambios negociados. Los ejemplos de Vigenère, AES, JavaFX y Unity corresponden al Reto 2. Los requisitos y entregables obligatorios siempre los establece el pliego de cada reto.

1. Objetivo de la actividad

CASO DE APLICACIÓN · EL RETO 2

Una solución integrada de cifrado que incluya Vigenère en Python, AES aplicado a ficheros, tratamiento correcto de archivos, un cliente de interacción JavaFX, un minijuego de descifrado en Unity 6.3, repositorio con PR y CI/CD, documentación y demostración. Es el alcance descrito en el Pliego Reto 02 (Conjunto), no una propuesta técnica nueva. La arquitectura concreta deberá justificarse en vuestro contrato técnico.

En cualquier proyecto profesional, desarrollar no consiste solamente en escribir código. También implica **comprender requisitos**, **dividir el trabajo**, **estimar el esfuerzo**, **priorizar**, **entregar incrementos**, **detectar bloqueos**, **colaborar** y **analizar los resultados**. En el Reto 2 practicaréis estas competencias por primera vez de manera estructurada.

La consigna principal es: planificar antes de desarrollar, registrar el trabajo mientras sucede y reflexionar al terminar. No se busca una planificación perfecta ni una memoria extensa, sino un proceso de trabajo honesto, ordenado y trazable.

Una breve historia: por qué apareció el desarrollo ágil

Durante décadas se popularizaron enfoques de desarrollo muy planificados y secuenciales: primero se especificaba todo, después se diseñaba, programaba, probaba y finalmente entregaba. Estos modelos pueden resultar adecuados cuando los requisitos están muy estables y el trabajo es predecible. El problema aparece cuando el cliente descubre nuevas necesidades, surgen dificultades técnicas o los supuestos iniciales resultan equivocados: si la validación llega al final, los errores pueden detectarse demasiado tarde.

A comienzos de los años noventa se consolidaron enfoques iterativos de desarrollo. Ken Schwaber y Jeff Sutherland presentaron Scrum públicamente en 1995. En 2001, diecisiete profesionales del software publicaron el Manifiesto por el Desarrollo Ágil: una declaración de cuatro valores y doce principios que prioriza las personas y la colaboración, el software que funciona, la relación con el cliente y la respuesta al cambio. Agile es una filosofía de trabajo; Scrum es uno de los marcos posibles para aplicarla. Kanban y las prácticas técnicas de XP son otros enfoques complementarios.

Agile no significa “trabajar sin plan” ni “hacer menos documentación”. Significa planificar en función de lo que sabemos, entregar incrementos verificables, obtener feedback y actualizar el plan cuando aprendemos algo nuevo. El contrato técnico sigue siendo importante: define el compromiso inicial y permite analizar después cualquier desviación.

Referencias: https://agilemanifesto.org/iso/es/manifesto.html · https://agilemanifesto.org/iso/es/principles.html · https://www.scrum.org/resources/scrum-development-process

VÍDEO BREVE · PRIMER CONTACTO CON SCRUM

«Scrum en 6 minutos» · Introducción visual a las iteraciones y las responsabilidades del equipo (recurso de apoyo, no sustituye a la Scrum Guide).
https://www.youtube.com/watch?v=HhC75IonpOU

¿Por qué funcionan estos métodos en proyectos reales?

1. REDUCEN EL RIESGO. Un incremento pequeño y probado permite descubrir problemas de integración, datos o requisitos antes de acumular semanas de trabajo equivocado.
2. HACEN VISIBLE EL PROGRESO. Un tablero actualizado, Issues cerradas y software funcionando muestran qué se ha conseguido. Estar “muy ocupado” no equivale necesariamente a entregar valor.
3. MEJORAN LA PRIORIZACIÓN. Si el tiempo es limitado, se desarrolla primero lo que más valor aporta o reduce más incertidumbre. Lo accesorio no bloquea lo esencial.
4. MEJORAN LA COORDINACIÓN. Las dailies permiten detectar dependencias y bloqueos; la programación en pareja pone decisiones y conocimiento técnico en común.
5. ENSEÑAN A ESTIMAR CON DATOS. Una estimación inicial es una hipótesis. Comparar horas previstas con esfuerzo real ayuda a ajustar la planificación de proyectos futuros.

6. CREAN UN CICLO DE APRENDIZAJE. Las reviews comprueban el producto; las retrospectivas analizan la manera de trabajar. La mejora debe traducirse en una acción observable.

Estas prácticas no garantizan el éxito por sí solas. Requieren objetivos claros, trabajo técnico de calidad, colaboración sincera y un seguimiento útil, no reuniones ceremoniales ni documentación creada a posteriori.

EJEMPLO DIDÁCTICO · RETO 2

Imaginad que desarrolláis primero la interfaz JavaFX y el minijuego de Unity, pero dejáis para el final comprobar que Vigenère descifra correctamente los mensajes con espacios o que AES permite recuperar un fichero binario sin alteraciones. Un fallo de cifrado podría obligaros a rehacer la integración. Una estrategia iterativa sería validar pronto el núcleo de cifrado y sus pruebas, integrar después los clientes y comprobar cada incremento antes de continuar. El objetivo no es programar menos: es descubrir los problemas cuando aún hay tiempo para solucionarlos. Esta secuencia es un EJEMPLO de planificación, no una distribución obligatoria de sprints.

POR QUÉ VALE LA PENA TRABAJAR ASÍ

Un estudio de 1.002 proyectos encontró una relación positiva entre enfoques ágiles, eficiencia y satisfacción de los interesados (Serrador y Pinto, 2015) [1]. En la encuesta State of Agile, los profesionales satisfechos con Agile también señalaron mejoras percibidas en colaboración, alineación con el negocio y calidad del software [6].

Son resultados observados y percepciones, no una garantía de que Scrum mejore automáticamente cualquier proyecto.

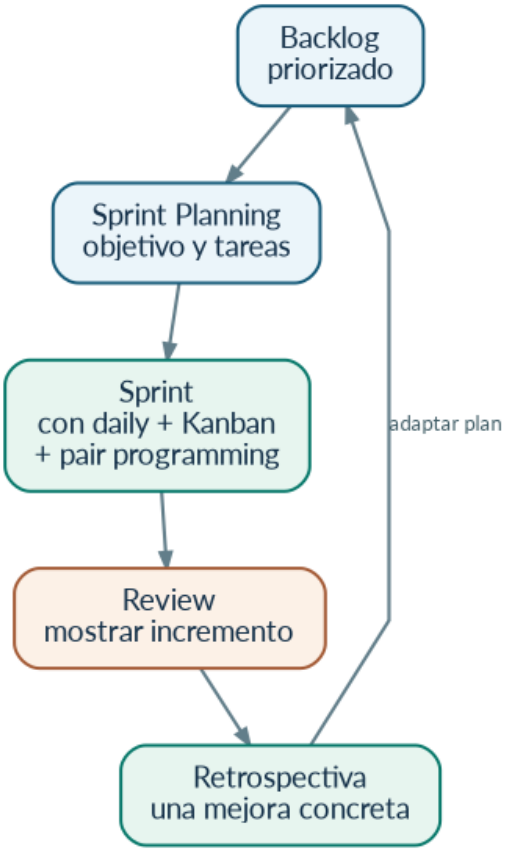
¿QUÉ VAMOS A COMPROBAR EN ADAT?

En el Reto 2 lo importante será comprobar si estimáis mejor, detectáis los bloqueos a tiempo, terminái tareas verificadas y ajustáis el plan cuando es necesario. Para demostrarlo utilizaréis GitHub Projects, las pruebas y los anexos de seguimiento.

Los informes y artículos originales quedan referenciados al final de la guía para quien quiera profundizar.

2. Scrum, Kanban y prácticas técnicas: qué es cada cosa

Scrum es un marco de trabajo ágil para abordar problemas complejos de forma iterativa e incremental. El equipo organiza el trabajo en sprints, entrega resultados utilizables, revisa lo conseguido y adapta su plan. Kanban es una forma visual de gestionar el flujo del trabajo mediante tarjetas y estados; aquí lo utilizaremos para representar las tareas de los sprints.



Esta guía combina tres elementos distintos: Scrum para organizar iteraciones y revisiones, Kanban para visualizar y limitar el trabajo en curso, y prácticas técnicas como programación en pareja y revisión de código para colaborar. No son sinónimos ni deben confundirse. En cada reto se ajustarán duración, número de integrantes y herramientas a las necesidades reales. En el Reto 2 utilizaremos una adaptación docente: parejas, una semana inicial de Discovery y dos sprints de una semana. En Scrum propiamente dicho el refinamiento es continuo, y sus responsabilidades y eventos no se reducen a este formato escolar.

VÍDEOS · DIFERENCIAR SCRUM, KANBAN Y SUS EVENTOS

- **«Scrum vs Kanban: diferencias y recomendaciones de uso»** · IEBS Digital School (2024). Útil para entender por qué utilizamos sprints y un tablero visual sin confundir ambos enfoques. https://www.youtube.com/watch?v=InE_500xdm0
- **«¿Qué es Scrum?»** · TodoCode, webinar en castellano. Selección opcional por capítulos: Sprint (20:11), planificación (27:52), Daily (35:53), Review (44:02), Retrospective (50:19). No es necesario verlo entero. https://www.youtube.com/watch?v=WZ8U_NHVdhI&t=1211s

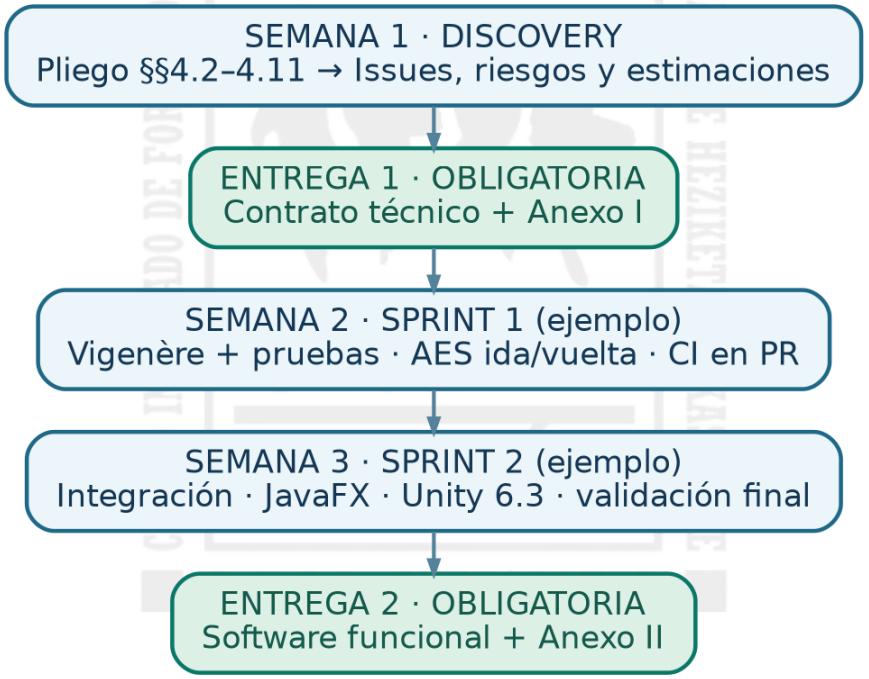
Conceptos que debéis conocer

- **Product Backlog**: conjunto priorizado de funcionalidades, mejoras y tareas pendientes.
- **Refinement**: aclarar requisitos, dividir elementos grandes, detectar dependencias y estimar.
- **Sprint**: periodo corto de trabajo con un objetivo concreto y un incremento verificable al final.
- **Sprint Planning**: reunión para decidir el objetivo y el trabajo que se abordará en el sprint.

- **Daily Scrum**: reunión breve de coordinación orientada a avanzar hacia el objetivo del sprint.
- **Sprint Review**: revisión y demostración del producto o incremento conseguido.
- **Sprint Retrospective**: reflexión sobre cómo ha trabajado el equipo y qué mejorará.
- **Definition of Done** (DoD): condiciones objetivas para considerar una tarea terminada.
- **Pair programming**: dos personas trabajan conjuntamente sobre el mismo problema, con roles driver y navigator, intercambiándolos periódicamente.

Ciclo general · analizar, planificar, construir, revisar y mejorar

El proceso puede repetirse en cualquier proyecto: analizar requisitos y riesgos, crear el backlog, planificar un incremento, desarrollarlo y verificarlo, revisar resultados y mejorar la forma de trabajar. En el Reto 2 este proceso se concreta en una semana inicial y dos sprints, con documentación en dos hitos. En futuros retos las fechas y los mínimos podrán ser diferentes.



3. Cómo organizar un proyecto (ejemplo de tres semanas)

Reto 2 · Semana 1 · Discovery y planificación

OBJETIVO: Convertir el pliego técnico en un plan realista, medible y compartido antes de desarrollar.

1. **Leer el pliego y aclarar requisitos.** Identificad funcionalidades y dependencias: Vigenère en Python (apartado 4.3), AES y ficheros (4.4-4.5), JavaFX (4.6 y 4.8), Git y CI/CD (4.7), documentación (4.9), Unity 6.3 (4.11) e integración (4.2). Anotad dudas y riesgos.
2. **Desglosar el trabajo en Issues.** Separad funcionalidades grandes en tareas que puedan desarrollarse y comprobarse; por ejemplo, cifrado Vigenère, descifrado, pruebas de caracteres especiales y pruebas de recuperación de ficheros AES.
3. **Definir qué significa terminar cada tarea.** Indicad criterios de aceptación, dependencias, responsables y estimación inicial en horas-persona. Una tarjeta «Hacer AES» es demasiado genérica.
4. **Calcular la capacidad disponible.** Estimad las horas de cada integrante durante los dos sprints y comparadlas con las estimaciones del backlog. Reservad tiempo para pruebas, integración y documentación.
5. **Priorizar y distribuir el backlog.** Fijad un objetivo para cada sprint y qué incremento podréis demostrar. La distribución del ejemplo de las semanas siguientes es orientativa: justificad vuestro propio plan.
6. **Preparar las reglas de colaboración.** Configurad GitHub Projects y sus estados; acordad la Definition of Done, cuándo haréis dailies, cómo organizaréis el pair programming y cómo registraréis bloqueos y cambios.

ENTREGA OBLIGATORIA AL CERRAR DISCOVERY: subid a la actividad del contrato técnico de Moodle un único PDF que incluya el contrato y su Anexo I (backlog, estimaciones, capacidad, sprints, acuerdos y riesgos). Conservad la versión inicial para poder compararla con el desarrollo real.

Reto 2 · Semana 2 · Sprint 1

OBJETIVO: Conseguir un primer incremento funcional y probado, no simplemente «tener muchas tareas empezadas».

1. **Celebrar el Sprint Planning.** Definid el objetivo de la semana, seleccionad Issues de acuerdo con vuestra capacidad y dejad constancia de las horas-persona previstas.
2. **Desarrollar el incremento.** Ejemplo orientativo: cifrado y descifrado Vigenère con casos de prueba de espacios y símbolos; primera prueba de ida y vuelta AES con un fichero. Podéis escoger otras prioridades si están justificadas.
3. **Integrar y validar durante el sprint.** Vinculad commits y Pull Requests a las Issues, revisad los cambios y comprobad las pruebas con CI cuando corresponda. No esperéis al último día para ejecutar las verificaciones.
4. **Coordinarse y registrar avances.** Al inicio de cada jornada trabajada, haced una daily breve (unos 5 minutos). Anotad qué se avanzó, próximas Issues, bloqueos y decisiones.
5. **Programar conjuntamente.** Realizad al menos dos sesiones de pair programming de 40-50 minutos, intercambiando driver y navigator y vinculando el registro al código producido.
6. **Actualizar las métricas.** Moved las tareas en el Kanban, registrad horas reales y horas restantes al finalizar cada jornada. Conservad los datos del burndown.
7. **Cerrar el sprint con dos actividades distintas.** Sprint Review: demostrad qué funciona realmente y qué queda pendiente. Retrospective: acordad al menos una mejora concreta que aplicaréis durante el Sprint 2.

RESULTADO AL CERRAR EL SPRINT 1: incremento demostrable, tablero e historial actualizados, dos sesiones de pair programming, registro de dailies, datos del burndown y primera retrospectiva. Estas evidencias se enlazarán en el Anexo II final.

Reto 2 · Semana 3 · Sprint 2 y entrega

OBJETIVO: Adaptar el plan con lo aprendido, terminar la integración y entregar un producto verificable junto a su seguimiento metodológico.

1. **Revisar y replanificar.** Contrastad las Issues previstas con las terminadas en el Sprint 1; identificad tareas arrastradas, bloqueos y capacidad real. Ajustad el Sprint Backlog sin borrar la estimación inicial.
2. **Completar el desarrollo prioritario.** Ejemplo orientativo: integrar las operaciones de cifrado con JavaFX (selección de ficheros, alertas y fatjar) y desarrollar o completar el minijuego Unity 6.3. El reparto técnico puede ser distinto si se justifica.
3. **Verificar la solución de extremo a extremo.** Comprobad que los flujos cumplen los criterios de aceptación, que los ficheros AES se recuperan íntegramente, que funcionan las pruebas y el pipeline, y que el producto se puede ejecutar y demostrar.
4. **Mantener el seguimiento.** Continuad con Kanban, dailies y registro de horas reales y restantes; completad el burndown del segundo sprint. Realizad otras dos sesiones de pair programming con intercambio de roles.
5. **Celebrar el Sprint Review final.** Mostrad el producto funcionando y diferenciad funcionalidades completadas, pendientes y limitaciones conocidas; no marquéis como Done algo que no cumpla su Definition of Done.
6. **Realizar la Retrospective final.** Comparad estimaciones y esfuerzo real, explicad los bloqueos más relevantes y verificad si aplicasteis la mejora acordada en el Sprint 1.
7. **Preparar el informe de metodología.** En el Anexo II enlazad el tablero, los registros de dailies y pair programming, los dos burndowns, las reviews, las retrospectivas y una breve reflexión de cada integrante.

ENTREGA OBLIGATORIA AL CERRAR EL SPRINT 2: adjuntad el Anexo II en PDF, con enlaces a evidencias de ambos sprints, en la actividad existente de documentación/entrega final de Moodle. Entregad el software y el resto de archivos técnicos en las actividades ya previstas en Moodle.



RECURSOS PRÁCTICOS · GITHUB PROJECTS, ISSUES Y PULL REQUESTS

- **«Planificación con GitHub Projects»** · recurso docente con demostración en vídeo y ejemplo paso a paso de Issues y tablero. Sirve como práctica guiada durante Discovery; comparad siempre las pantallas con la versión actual de GitHub. https://iesataulfoargentasor.github.io/sbd_2026_2027_new/ut1/planificacion/

- Vídeo de tablero e Issues enlazado desde la guía docente: https://www.youtube.com/watch?v=7eeHBaPnUGM
- Tutorial oficial en castellano para configurar Projects, iteraciones y automatizaciones (referencia actualizada): https://docs.github.com/es/issues/planning-and-tracking-with-projects/learning-about-projects/quickstart-for-projects
- **«Pull Requests en GitHub»** · MoureDev, fragmento seleccionado del curso de Git y GitHub (3:30:46). Útil para comprobar cómo enlazar el trabajo del tablero con cambios revisables. https://www.youtube.com/watch?v=3GymExBkKjE&t=12646s

PROFESIONAL C.I.F.P. CIUDAD JARDÍN L.H.I.I. INTEGRATUA
CENTRO INTEGRADO DE FORMACION LANBIDE HEZIKETAKO IKASTETXE

4. Herramientas y reglas generales

- GitHub será la fuente de verdad para tareas, decisiones y evidencias: Issues, Pull Requests y GitHub Projects. Los anexos en PDF deben resumir la planificación y los aprendizajes, y enlazar la actividad real del repositorio, sin copiar ni duplicar registros. En otros contextos profesionales podrían utilizarse herramientas equivalentes, como Jira o Azure DevOps.
- Estados mínimos del tablero: Backlog, Ready o TO-DO, In Progress, Review/Test, Blocked y Done. Podéis añadir estados si los justificáis.
- Cada trabajo relevante debe existir como Issue identificable; cada cambio de código debe poder relacionarse con una Issue y, cuando corresponda, con una Pull Request.
- Como primer sistema de estimación utilizaremos horas-persona; los story points pueden presentarse después como una alternativa relativa, no como una conversión directa a horas. Distinguid esfuerzo y tiempo transcurrido: una sesión de 1 hora con 2 personas implica 2 horas-persona, mientras que su duración real sigue siendo de 1 hora. Adaptad la capacidad según el número de participantes.
- Recomendación: tareas de 1 a 4 horas-persona cuando sea razonable; dividid las tareas excesivamente grandes, evitando fragmentaciones artificiales.
- Limitad el trabajo en curso (WIP) de forma coherente con el tamaño del equipo y las dependencias. Evitad empezar numerosas tareas a la vez: terminar y validar incrementos suele ofrecer más información que mantener muchas tareas a medias.
- Podéis modificar el backlog y reestimar si aprendéis algo nuevo, pero debe quedar constancia de qué cambió, cuándo y por qué. No borréis la planificación original.
- Toda evidencia debe generarse durante el trabajo. Capturas y declaraciones finales pueden ayudar a explicar, pero no sustituyen el historial de Issues, comentarios, commits y PR.

RETO 2 · ENTREGA INICIAL OBLIGATORIA

En el RETO 2, el Anexo I se integra en el mismo PDF del contrato técnico y se sube a la actividad del contrato técnico que ya existe en Moodle, antes de comenzar el Sprint 1. Debe reflejar la planificación original, no reconstruirse después. No se crea otra entrega para el anexo. En otros retos prevalecerá el pliego.

5. Documento inicial de planificación (aplicación al Reto 2)

En cualquier proyecto conviene registrar una línea base: qué se pretende construir, cómo se divide y qué esfuerzo se prevé. Para el RETO 2 será OBLIGATORIO incorporar al documento del contrato técnico el «Anexo I. Plan de ejecución, estimaciones y sprints». Ambos forman un solo archivo PDF, que se entrega en Moodle al finalizar Discovery y antes del Sprint 1. Este anexo metodológico no debe confundirse con el «Anexo I - ADAT» del pliego, dedicado a los resultados de aprendizaje.

Contenido mínimo del Anexo I (obligatorio)

A. **Objetivo y alcance**: breve resumen del producto que vais a construir, funcionalidades esenciales y prioridades del proyecto.
B. **Backlog desglosado**: enlace al tablero y relación de Issues priorizadas. Cada tarea incluirá título, descripción, criterios de aceptación verificables, estimación en horas-persona, responsable(s), sprint previsto y dependencias, si existen.
C. **Capacidad disponible**: horas aproximadas de trabajo de cada miembro durante los dos sprints, indicando supuestos relevantes y distinguiendo capacidad del equipo de las horas estimadas de las tareas.
D. **Plan de sprints**: objetivo del Sprint 1 y del Sprint 2, funcionalidades previstas para cada uno y justificación del orden. El Sprint 2 será una previsión que podrá revisarse tras la primera retrospectiva.
E. **Acuerdos de equipo**: Definition of Done, política de actualización del Kanban, momento de las dailies, organización de pair programming y criterio para comunicar bloqueos.
F. **Riesgos y dependencias**: al menos los riesgos relevantes detectados y una acción de prevención o respuesta para cada uno.

El Anexo I no puede consistir únicamente en pegar un enlace al Kanban: debe explicar qué pretendéis hacer, cómo lo habéis descompuesto y por qué vuestras estimaciones y prioridades son razonables. Las Issues contienen el detalle operativo.

Del pliego al Product Backlog: ejemplo orientativo de Issues

Así se convierte un pliego en un Product Backlog: cada viñeta representa una posible Issue de GitHub Projects y su estimación FICTICIA en horas-persona. A y B son dos integrantes de ejemplo: la letra identifica a quien coordina la tarea, no a la única persona que puede contribuir. «A + B» indica trabajo compartido. Repartid la carga según la capacidad del equipo y revisad las asignaciones en cada Sprint Planning; en equipos mayores podría existir también una persona C. En otros retos se aplica el mismo método con tareas diferentes.

Vigenère (Python) | Apartado 4.3

- Implementar las funciones de cifrado y descifrado - 3 h-persona · A
- Definir cómo se tratan espacios y símbolos - 2 h-persona · B
- Crear pruebas de ida y vuelta (cifrar y recuperar el original) - 2 h-persona · A + B

AES y ficheros | Apartados 4.4 y 4.5

- Implementar cifrado y descifrado de ficheros - 4 h-persona · B
- Verificar recuperación byte a byte, rutas y errores - 3 h-persona · A
- Analizar el uso de memoria con archivos grandes - 2 h-persona · A + B

Git y CI/CD | Apartado 4.7

- Configurar un pipeline que valide automáticamente una Pull Request - 2 h-persona · A + B
- Aplicar ramas, PR e identidades Git - práctica continua · A + B

Cliente JavaFX | Apartados 4.6 y 4.8

- Seleccionar ficheros de entrada y salida y mostrar alertas - 4 h-persona · A
- Integrar las operaciones de cifrado en el cliente - 4 h-persona · B
- Generar el fatjar y comprobar su ejecución en las plataformas previstas - 3 h-persona · A

Unity 6.3 | Apartado 4.11

- Mostrar texto cifrado, recoger y validar respuestas en una escena 2D - 4 h-persona · B
- Programar el avance del enemigo y los estados de victoria y derrota - 4 h-persona · A

Integración y documentación | Apartados 4.2 y 4.9

- Verificar un flujo completo entre componentes - 4 h-persona · A + B
- Preparar manuales, decisiones y evidencias de recuperación - 3 h-persona · B

Las cifras anteriores son solo un EJEMPLO, no horas oficiales. No sustituyen los requisitos técnicos ni una planificación real. En cualquier reto habrá que detectar dependencias, integración con otros módulos y criterios de aceptación antes de considerar una función terminada.

Criterio de Done adaptado al Reto 2: la funcionalidad se demuestra; se comprueba con pruebas o verificaciones reproducibles; la PR queda revisada e integrada; el pipeline muestra el resultado cuando proceda; y se actualiza la documentación afectada. Para Vigenère debe constar el criterio sobre caracteres especiales, y para AES la recuperación del contenido original.

Riesgos que sí tienen sentido aquí: corrupción al recuperar ficheros binarios; reglas ambiguas de caracteres en Vigenère; empaquetado JavaFX entre Linux/Windows; integración de componentes; fallos del pipeline y lógica de colisiones o estados en Unity. En Discovery indicad cómo detectaríais cada riesgo antes de la entrega final.

Ejemplo de tarea correctamente definida

Issue #12 · Descifrar mensajes con Vigenère (ejemplo ficticio)

- **Referencia:** apartado 4.3 del pliego
- **Sprint previsto: Sprint 1 · Estimación inicial: 3 h-persona**
- **Asignación:** Persona A (responsable de la Issue); Persona B (colaboración y revisión)
- **Dependencia:** Issue #7, definición del tratamiento de espacios y símbolos
- **Descripción:** Implementar el descifrado de un mensaje utilizando la clave proporcionada, de acuerdo con las reglas documentadas para espacios, tildes y símbolos.
- **Criterios de aceptación:**
    - Cifrar y después descifrar con la misma clave recupera el texto original según las reglas acordadas.
    - El comportamiento con espacios, símbolos y caracteres especiales está definido y documentado.
    - Existen pruebas automatizadas para los casos normales y los casos especiales.
    - La Pull Request está revisada y la validación de CI puede consultarse en GitHub.
- **Nota:** el pliego no exige crear un endpoint REST para esta funcionalidad.

Línea base de planificación

Al entregar el contrato, guardad una copia versionada del Anexo I en el repositorio (por ejemplo, docs/metodologia/anexo-I-planificacion.md). Esa versión será vuestra planificación inicial de referencia. Durante el reto podréis actualizar el tablero, pero las diferencias se explicarán en el informe final.

6. Prácticas de seguimiento durante los sprints

6.1. Sprint Planning

Al comenzar cada sprint, dejad por escrito el **objetivo**, el **conjunto de Issues** seleccionadas y las **horas-persona previstas**. En el Sprint 2, revisad el plan con los resultados reales del Sprint 1. El plan debe ser alcanzable, no una lista de deseos.

6.2. Kanban y trazabilidad

Actualizad el estado de las tareas mientras trabajáis. Al cerrar una Issue, registrad el esfuerzo real, una referencia al código/PR correspondiente (con GitHub debería estar auto) y la confirmación de los criterios de aceptación. Si surge un bloqueo, señaladlo en el tablero e indicad su causa y cómo vais a resolverlo.

6.3. Daily breve, con registro

Como práctica general, una daily es una breve coordinación del equipo para inspeccionar el progreso hacia el objetivo del sprint, identificar impedimentos y ajustar el trabajo próximo; no debe convertirse en un informe burocrático. En el RETO 2 se requiere **una reunión de unos 5 minutos en cada jornada** en la que se trabaje, con un registro conjunto en una Issue o archivo versionado. No se inventarán reuniones de días sin actividad.

- Plantilla de daily: Fecha · Asistentes · Progreso desde la última sesión (Issues) · Próximas tareas (Issues) · Bloqueos · Decisiones o cambios en la planificación.
- Ejemplo ficticio: Día 3 · A y B · Ayer cerramos las pruebas básicas de Vigenère (#8, PR #14). Hoy abordamos el descifrado de textos con signos (#12). Bloqueo: no hemos definido qué hacer con las tildes y los espacios. Decisión: documentar la regla, fijar casos de prueba y actualizar la Issue antes de programar.

6.4. Pair programming · colaboración técnica

El pair programming es una práctica de programación colaborativa, no un componente obligatorio de Scrum. Dos personas trabajan conjuntamente sobre un problema: driver escribe el código y navigator revisa alternativas, riesgos y decisiones; ambos intercambian los roles. Es útil para compartir conocimientos y revisar problemas complejos.

- Requisito concreto del RETO 2: **al menos DOS SESIONES por sprint (CUATRO en total) de 40–50 minutos, con rotación de roles**. Para proyectos individuales u otros tamaños de equipo se podrán plantear revisiones de código u otras formas de colaboración según el pliego.

Cuando se exija pair programming, cada sesión debe dejar evidencias: fecha, duración, Issue, participantes y roles, intercambio de funciones, decisión técnica, resultado y enlace a código o PR. La coautoría de commits puede complementar el registro, pero no demuestra por sí sola que los dos hayan trabajado juntos.

Ejemplo ficticio de registro de pair programming (PP-02)

Sprint: 1 · **Duración:** 45 minutos · **Issue:** #16, recuperar ficheros AES

Rotación de roles

- **Minutos 0 a 22:** Persona A (driver) y Persona B (navigator).

- **Minutos 22 a 45:** Persona B (driver) y Persona A (navigator).

Decisión técnica: comparar byte a byte el fichero original y el recuperado, no solo comprobar que puede abrirse.

Resultado: prueba de ida y vuelta añadida y Pull Request #18 revisada.

Evidencias para un registro real: fecha de la sesión, enlace a la Issue y a los commits o PR asociados.

En el RETO 2 ambos miembros deberán pasar por los dos roles en cada sprint. El principio general vale para cualquier equipo: colaborar significa intervenir técnicamente, contrastar decisiones y compartir responsabilidad, no limitarse a observar.

6.5. Medición del progreso

La medición del flujo puede incluir tiempo de ciclo, tareas finalizadas, bloqueos, horas estimadas frente a reales y evolución del trabajo pendiente. Un burndown muestra el trabajo que queda; un burnup muestra lo terminado y, si cambia, el alcance total. No son las únicas métricas de Agile ni deben utilizarse para comparar personas. En el RETO 2 registrad las horas-persona pendientes al terminar cada jornada de trabajo y elaborad un burndown de cada sprint, con curva ideal, real y datos de respaldo.

Si surgen tareas o cambia una estimación, documentadlo; una curva que sube puede ser completamente razonable. Las horas restantes son la mejor estimación actual, no necesariamente las horas iniciales menos las ya dedicadas. Podéis generar el gráfico en una hoja de cálculo o utilizar una herramienta equivalente; incluid la imagen o enlace verificable en el Anexo II.

Burndown de ejemplo aplicado a AES (DATOS FICTICIOS)

Situación inicial

Una pareja estima 16 horas-persona para las tareas AES del Sprint 1. Al acabar cada jornada apunta cuántas horas-persona cree que FALTAN para terminarlas, no cuántas ha trabajado.

Trabajo pendiente (datos ficticios)

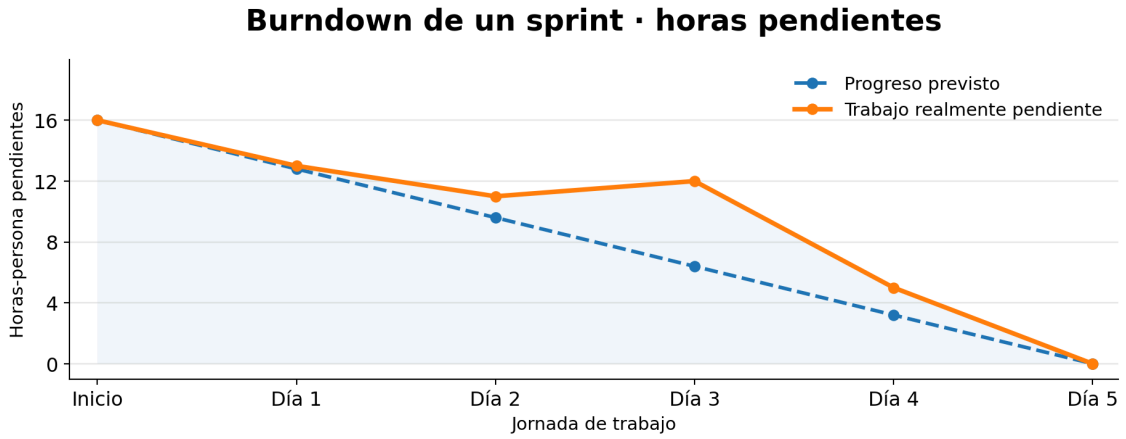
- **Inicio:** 16 h pendientes.
- **Días 1 y 2:** 13 h y 11 h pendientes.
- **Día 3:** 12 h pendientes. Se descubre un error al recuperar ficheros binarios.
- **Días 4 y 5:** 5 h y 0 h pendientes.

¿Por qué sube la curva el día 3?

Aunque el equipo ha trabajado, descubre que necesita una prueba y una corrección adicionales. Por eso revisa el cálculo: ahora quedan 12 h, una más de las 11 h estimadas el día anterior.

Qué debéis aprender

Un aumento del trabajo pendiente no demuestra que el sprint haya fracasado. El burndown sirve para visualizar avances, dificultades y reestimaciones. En vuestro Anexo II utilizaréis datos reales y explicaréis cualquier cambio relevante.



EJEMPLO FICTICIO · El día 3 aumenta el trabajo pendiente al detectar un problema con un fichero binario.

VÍDEO · CÓMO HACER UNA RETROSPECTIVA ÚTIL

«La retrospectiva en Scrum» · curso de fundamentos en castellano (clase 9). Tras verlo, cada equipo debería poder identificar un problema de su sprint, su causa y una acción concreta y verificable para el siguiente. https://www.youtube.com/watch?v=D1CJUiYWhqM

6.6. Sprint Review y Retrospective

Al final de cada sprint registrad dos cosas diferenciadas. **Review**: qué incremento funciona y puede demostrarse, qué queda sin terminar y qué criterios de aceptación cumple. **Retrospective**: qué salió bien, qué os frenó, qué cambiaréis y quién se compromete a aplicar la mejora.

La retrospectiva del Sprint 1 debe proponer al menos UNA ACCIÓN DE MEJORA concreta para el Sprint 2; en la retrospectiva final se comprobará si se aplicó y con qué resultado.

Ejemplo de Sprint Review y Retrospective (ficticio)

Review del Sprint 1: demostramos en Python que Vigenère cifra y descifra textos con los casos acordados y que un archivo AES se recupera sin alteraciones; adjuntamos las pruebas y las PR. Todavía no está integrado el cliente JavaFX: queda indicado como trabajo pendiente, no como Done.

Retrospective del Sprint 1: estimamos poco tiempo para probar distintos tipos de fichero; en Sprint 2 acordamos definir casos de prueba antes de desarrollar la integración. Al terminar, comprobaremos si esa acción evitó correcciones tardías. La review responde «¿qué funciona?»; la retrospective responde «¿cómo podemos trabajar mejor?».

6.7. Feedback del cliente, MVP y gestión de cambios

En un proyecto real, el cliente puede descubrir necesidades nuevas al probar un prototipo. Una Sprint Review sirve también para recoger ese feedback, decidir prioridades y ajustar el Product Backlog. Aprenderemos a hacerlo de manera progresiva durante el curso, no mediante más formularios.

Lean y MVP: un producto mínimo viable (MVP) es la versión más pequeña que permite comprobar una hipótesis con usuarios y aprender de su respuesta. Lean propone reducir el trabajo que no aporta valor. No significa entregar software sin probar ni omitir requisitos del pliego. En el Reto 2, comprobar que AES recupera íntegramente un fichero es un prototipo técnico verificable; no es por sí solo un MVP validado con clientes.

¿Cómo responder a una nueva petición?

1. Registrar la solicitud y su motivo en una Issue de GitHub.
2. Estimar qué cambia: esfuerzo, dependencias, riesgos, pruebas y trabajo comprometido.
3. Negociar si se prioriza, aplaza o descarta, y qué otras tareas se verían afectadas.
4. Dejar la decisión justificada en la Issue y actualizar el backlog sin borrar el plan inicial.

EJEMPLO FICTICIO · RETO 2

Tras probar un prototipo AES, el cliente pide poder cifrar carpetas enteras. El equipo estima 6 horas-persona adicionales. Decide mantener el objetivo del Sprint 1, anota la solicitud en el backlog y propone revisar su prioridad en el siguiente Sprint Planning. La petición no reemplaza las funcionalidades obligatorias del pliego.

APLICACIÓN PROGRESIVA · SIN NUEVAS ENTREGAS

En el Reto 2 se puede comentar un cambio hipotético durante la Review, sin exigir otro anexo, otra actividad de Moodle ni implementar esa petición. En retos posteriores podrán introducirse clientes simulados, MVP y cambios negociados, si se acuerdan con el profesorado y el pliego lo permite.

RETO 2 · ENTREGA FINAL OBLIGATORIA

En el RETO 2, el Anexo II se entrega en formato PDF con la documentación final, en la actividad correspondiente que ya está habilitada en Moodle. El software se entrega en su espacio previsto. Sin el PDF del Anexo II y sus enlaces a evidencias de ambos sprints no se acredita el seguimiento exigido. No es necesaria una nueva actividad de Moodle.

7. Informe final de seguimiento y retrospectiva

Un informe final útil contrasta el plan original con el trabajo realizado, sin duplicar el historial del repositorio. En el RETO 2 presentaréis el **«Anexo II. Seguimiento y retrospectiva del desarrollo»** como un PDF, junto a la documentación final en la entrega ya disponible en Moodle. El software se entregará en su actividad correspondiente. Extensión orientativa del anexo: 2–3 páginas de análisis, más gráficos y enlaces a evidencias. Su estructura puede reutilizarse en próximos retos.

Ejemplo de informe final · Reto 2 (datos ficticios)

«Para la integración de AES con JavaFX estimamos 4 h-persona y empleamos 6 porque las rutas de entrada/salida no se validaban correctamente. Documentamos el bloqueo en la daily del día 3 y añadimos comprobaciones y alertas. En el Sprint 2 aplicamos la mejora acordada: escribir casos de prueba al abrir cada Issue. El cambio queda acreditado en las Issues, PR y el burndown». Este análisis con causas y enlaces vale más que una lista de actividades sin reflexión.

Contenido mínimo del Anexo II (obligatorio)

1. **Referencias**: enlace al repositorio, al tablero GitHub Projects y a la versión inicial del Anexo I.
2. **Resumen por sprint**: objetivo, tareas previstas y terminadas, tareas arrastradas, esfuerzo estimado frente al real y principales cambios de alcance.
3. **Seguimiento verificable**: enlace al historial de dailies, tabla de bloqueos relevantes y decisiones tomadas, y dos gráficos burndown con sus datos.
4. **Colaboración técnica**: enlaces a PR y revisiones; en el RETO 2, los cuatro registros mínimos de pair programming, con evidencias de roles e Issues asociadas.
5. **Reviews y retrospectivas**: qué se demostró al terminar cada sprint, qué funcionó y qué no, qué mejora se acordó tras el Sprint 1 y evidencia de su aplicación (o explicación de por qué no se pudo aplicar).
6. **Comparación con el contrato inicial**: qué habíais planificado, qué sucedió realmente y qué estimaríais o organizaríais de otra manera en un próximo proyecto.
7. **Reflexión sobre las aportaciones**: en proyectos grupales, contribución de cada integrante y principal aprendizaje; para el RETO 2, experiencia de ambos participantes como driver/navigator.

No copiéis todas las dailies ni todas las Issues al informe: enlazadlas. El Anexo II debe INTERPRETAR las evidencias, no duplicarlas ni maquillar el resultado.

8. ¿Cómo se evaluará el proceso?

Rúbrica orientativa para valorar el trabajo con metodologías ágiles en el Reto 2. Se evaluará lo que el equipo haya planificado, registrado y demostrado con evidencias, no la ausencia de problemas.

Criterio	0–4,9 Insuficiente	5–7,4 Básico	7,5–10 Destacado
Planificación Anexo I	Sin backlog coherente; faltan estimaciones, criterios o plan de sprints.	Backlog y sprints definidos; estimaciones o prioridades poco justificadas.	Issues verificables, capacidad realista, prioridades y riesgos justificados.
Seguimiento Kanban y dailies	Tablero desactualizado; dailies ausentes o reconstruidas al final.	Tablero y dailies regulares; registra los cambios y bloqueos principales.	Seguimiento continuo y trazable; bloqueos y reajustes explicados a tiempo.
Colaboración Pair programming y PR	Faltan sesiones, rotación de roles o evidencias de revisión.	Cuatro sesiones registradas con cambio de roles y enlaces a código o PR.	Cuatro sesiones verificables, decisiones compartidas y revisiones útiles.
Análisis final Burndown y Anexo II	Faltan gráficos o retrospectivas; el anexo no contrasta plan y realidad.	Dos burndowns, reviews y retrospectivas; compara plan y resultados de forma básica.	Datos contrastados, causas explicadas, mejora comprobada y reflexión individual.

Cómo se aplicará: para cada criterio se asignará un nivel y una calificación dentro de su tramo, según las evidencias disponibles. Esta rúbrica describe la calidad del proceso, pero no fija su peso en la nota total del reto.

Importante: equivocarse al estimar o cambiar de sprint no penaliza por sí mismo. Sí se valora explicar los cambios a tiempo. El profesorado contrastará los anexos con el historial de Issues, commits y Pull Requests; podrá solicitar aclaraciones y valorar las contribuciones individuales.

9. Checklist reutilizable · mínimos del Reto 2

Hito 1 · Al finalizar Discovery

- ☐ Contrato técnico completo en un solo PDF, con el Anexo I incorporado; subida a la actividad existente de Moodle.
- ☐ Comprobad que el Anexo I del PDF incluya alcance, backlog, estimaciones en horas-persona, capacidad, plan de sprints, acuerdos y riesgos.
- ☐ Tablero de GitHub Projects operativo e Issues con criterios de aceptación.
- ☐ Versión inicial del plan guardada en el repositorio.

Hito 2 · Al finalizar el Sprint 2

- ☐ Software y documentación técnica entregados en sus actividades de Moodle ya habilitadas.
- ☐ Historial de dailies de las jornadas efectivamente trabajadas.
- ☐ Reto 2: dos sesiones de pair programming por sprint, con rotación y código asociado; en otros retos, aplicar la práctica colaborativa acordada.
- ☐ Tareas cerradas con esfuerzo real y evidencia de cumplimiento.
- ☐ Dos seguimientos/gráficos burndown y datos de respaldo.
- ☐ Dos reviews y dos retrospectivas, con mejora aplicada o justificada.
- ☐ Anexo II en PDF con comparación plan-realidad y aportaciones de cada participante; adjuntado a la entrega de documentación final en Moodle.

Criterio de revisión: evidencias, no declaraciones

El profesorado revisará conjuntamente el Anexo I (línea base), el historial real de GitHub (fechas de Issues, comentarios, PR y commits) y el Anexo II (análisis final). No es obligatorio que el profesorado observe presencialmente cada daily ni cada sesión de pair programming; por eso es imprescindible mantener evidencias oportunas y trazables. Los registros deben ser honestos: una desviación explicada aporta más aprendizaje que un plan artificialmente perfecto.

10. Material de consulta

- Pliego técnico que debéis utilizar como fuente de requisitos: https://docs.google.com/document/d/1K5wRflWcBlr4WeeQhcH8ci1LVCWyvT-EXhXwD8om2iE/edit
- Apartados especialmente relevantes: §4.2 integración; §4.3 Vigenère Python; §4.4–4.5 AES y archivos; §4.7 PR/CI/CD; §4.8 JavaFX; §4.9 documentación; §4.11 Unity. El apartado §4.10 de la versión consultada no concreta funcionalidades: **cualquier duda sobre ese alcance se debe aclarar durante Discovery, nunca suponerla**.
- Guía oficial de Scrum (Schwaber y Sutherland): https://scrumguides.org/scrum-guide.html
- Introducción a los eventos de Scrum: https://www.scrum.org/resources/learn-more-about-each-scrum-event
- Documentación de GitHub Projects: https://docs.github.com/es/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects

ARTÍCULOS E INFORMES TÉCNICOS CITADOS

01. Serrador, P. y Pinto, J. K. (2015). Does Agile work? International Journal of Project Management. https://doi.org/10.1016/j.ijproman.2015.01.006
02. DORA / Google Cloud (2023). Accelerate State of DevOps Report. https://dora.dev/research/2023/dora-report/
03. Hannay, J. E. et al. (2009). The effectiveness of pair programming: A meta-analysis. Information and Software Technology. https://doi.org/10.1016/j.infsof.2009.02.001
04. DORA (2018). Accelerate State of DevOps Report. https://dora.dev/research/2018/dora-report/
05. Jørgensen, M. (2024). Systematic literature review on the front-end phase of agile software development projects. https://doi.org/10.1016/j.jss.2024.112155
06. Digital.ai (2024). 17th State of Agile Report. Encuesta publicada en enero de 2024 (788 profesionales). Se puede consultar la fuente original en https://digital.ai/resource-center/analyst-reports/state-of-agile-report/.

Lectura crítica: los estudios correlacionales, encuestas de percepción y comparaciones entre grupos no prueban automáticamente causalidad. Los resultados describen los contextos y métricas de cada publicación; no garantizan resultados idénticos en clase ni miden el efecto aislado de Scrum.

Esta guía propone un método de trabajo para todo el curso. El Reto 2 introduce las prácticas básicas y sus dos anexos; los siguientes retos permitirán aplicarlas con mayor autonomía e incorporar gradualmente prototipos, feedback del cliente y decisiones sobre cambios cuando así se acuerde en sus pliegos. Ningún ejemplo, estimación o cambio ficticio de esta guía modifica los requisitos técnicos ni crea entregas adicionales.